

integration_sec08

8. Phase 7: Testing Strategy and Anti-Bluff Quality Assurance

Phase 7 closes the 56.4% coverage gap between the current 43.6% and the 100% target across all packages touched by the LLMsVerifier integration. The preceding phases produced eight pipeline steps in `internal/verifier/pipeline.go`, five scoring components in `internal/verifier/scoring/engine.go`, a 3-tier discovery system, four selection strategies, and four new API endpoints. This phase defines the testing strategy, implements unit and integration tests for every component, establishes the anti-bluff Challenge test framework, and sets performance SLA targets. Every test must prove real behavior; mocks are permitted only for external HTTP calls.

8.1 Testing Strategy Overview and Coverage Targets

8.1.1 Test Pyramid Architecture

The test pyramid allocates 70% of test files to unit tests for algorithmic correctness, 20% to integration tests for behavioral validation across package boundaries, and 10% to end-to-end tests for complete user journeys. Table 8.1 defines each layer.

Table 8.1: Test Pyramid — Layer Definitions and Coverage Targets

Layer	Test Files	Coverage Target	Execution	Anti-Bluff Rule
Unit (70%)	~140	95% per package	< 30s	Mocks only for external HTTP; system under test uses real impl
Integration (20%)	~40	85% path coverage	< 5 min	Real infrastructure: SQLite, Dockerized services, JWT tokens
E2E (10%)	~20	100% user journey	< 10 min	Full system stack; tests run against built binary

Unit tests target the 15+ new integration files. Every branching function — `calculateResponseSpeedScore` with its four throughput tiers, `Allow` in `middleware.go` with its sliding-window algorithm,

CalculateComprehensiveScore with five-component weighted aggregation — receives dedicated table-driven tests exercising each branch. The 95% coverage target applies per package, not globally; global averaging is prohibited because it allows one well-tested package to mask an untested one.

Integration tests validate cross-package composition. The scoring engine calls `se.dbIntegration.UpdateModelScores()` after computing the weighted total; an integration test verifies this write persists and subsequent reads return the stored value. The discovery service falls back from static registry to provider API to `models.dev`; an integration test verifies tier-1 failure triggers tier-2 within the configured timeout. E2E tests validate complete journeys: discovery populates the registry, the gatekeeper filters by threshold, the selector picks the top model, the API returns it, and the translation pipeline produces valid output.

8.1.2 Coverage Gap Analysis

Current coverage across the seven affected packages is 43.6%. Table 8.2 breaks down the gap.

Table 8.2: Coverage Gap Analysis by Package

Package	Current	Target	Gap	Priority	Blocker
pkg/ translator/ llm/	43.6%	100%	+56.4%	High	61 disabled tests in <code>provider_test.go</code> must be re-enabled for verified provider flow
internal/ config/	~60%	100%	+40%	High	New verifier config structs (<code>VerifierConfig</code> , <code>DiscoveryConfig</code> , <code>ScoringConfig</code>) untested
pkg/api/	~50%	100%	+50%	Critical	Four new endpoints (<code>/api/v1/models/discover</code> , <code>/verify</code> , <code>/select</code> , <code>/scores</code>) have zero coverage
pkg/ verification/	~30%	100%	+70%	Critical	8-step pipeline flow has no automated validation
pkg/events/	~40%	100%	+60%	Medium	Verifier event bus integration lacks tests
internal/ verifier/ (new)	0%	100%	+100%	Critical	All new files — <code>client.go</code> , <code>pipeline.go</code> , <code>scoring/engine.go</code> , <code>discovery/*.go</code> , <code>selection/*.go</code> — untested
internal/ services/	~45%	100%	+55%	High	<code>llmsverifier_score_adapter.go</code> field mapping and cache logic untested

The 61 disabled tests in `pkg/translator/llm/provider_test.go` are the largest blocker. They were disabled during initial integration because they relied on the legacy provider factory. Re-enabling them requires updating the test setup to initialize the verifier client: replace `NewProvider(cfg)` with `NewProvider(cfg, WithVerifierClient(verifierClient))`, where `verifierClient` connects to a dockerized LLMsVerifier via `testcontainers`.

8.1.3 Testing Philosophy: The Anti-Bluff Mandate

The anti-bluff philosophy is codified in eight constitution rules. The project experienced a failure mode where all tests passed but features did not work — caused by over-mocking. Tests verified that `mockClient.Verify()` returned expected values, but never verified that the real HTTP client could connect, that the real database could persist, or that the scoring engine produced numerically correct results.

Rule T-01: a passing test guarantees the feature works for end users. A score adapter test cannot assert that `adapter.Convert()` was called with correct arguments; it must assert that the converted `ProviderScore` contains the correct `OverallScore` from a live `LLMsVerifier` instance. Rule T-02: mocks restricted to external HTTP/API calls only. The `modelsDevClient` in `scoring/engine.go` may be mocked because `models.dev` is external; the `DatabaseIntegration` struct must not be mocked — tests use `database.New(":memory:").` Rule T-03: every test verifies real behavior, not just invocation. Calling `engine.CalculateComprehensiveScore()` and asserting only `require.NoError(t, err)` violates this rule; the test must also assert the score falls within the expected range given the input model characteristics. Rule T-04: integration tests use real infrastructure. The `tests/integration/` directory contains a `test-compose.yml` with a real `LLMsVerifier` container, SQLite on a volume mount, and mock provider endpoints via `mockserver/mockserver`.

Rules T-05 through T-08 are enforcement mechanisms: T-05 declares a feature without a Challenge test incomplete; T-06 requires Challenge tests to run in production-like environments; T-07 makes coverage below 100% a build-blocking failure; T-08 imposes a 24-hour fix-or-remove deadline on flaky tests.

8.1.4 Constitution Rule Documentation

The anti-bluff rule enters three governance documents. In `internal/verifier/CONSTITUTION.md`, Rule T-01 reads: "A passing test guarantees the feature works for end users — mocks in production paths are prohibited." Enforcement: `go test -cover` plus manual review; violation: PR rejection. In `CLAUDE.md`, the Testing Guidelines section specifies: when testing `CalculateComprehensiveScore`, use a real `ScoringEngine` with `DatabaseIntegration` backed by `:memory: SQLite`; mock only `ModelsDevClientInterface`. In `AGENTS.md`, the Testing Agent's decision tree starts with: on test failure, check if the test mocks the system under test → if yes, rewrite with real implementations → if still failing, investigate production code.

8.2 Unit Testing — Core Verification Components

8.2.1 Verifier Client Test Implementation

The verifier client in `internal/verifier/client.go` wraps HTTP communication with `LLMsVerifier`. Its constructor `func NewVerifierClient(cfg *Config) (*Client, error)` validates configuration and verifies connectivity via a health ping. The test file `internal/`

verifier/client_test.go targets 95% coverage. Container-based tests are gated behind //go:build verifier_integration for CI only; fast-path tests use httptest.Server.

Code Block 8.1: internal/verifier/client_test.go

```
package verifier

import (
    "context"
    "errors"
    "net/http"
    "net/http/httptest"
    "strings"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestNewVerifierClient(t *testing.T) {
    tests := []struct {
        name      string
        cfg       *Config
        wantErr   bool
        errContains string
        validate  func(t *testing.T, c *Client)
    }{
        {
            name: "valid config with explicit timeout",
            cfg: &Config{
                APIURL:  "https://verifier.example.com",
                APIKey:  "test-key-123",
                Timeout: 10 * time.Second,
            },
            validate: func(t *testing.T, c *Client) {
                assert.Equal(t, "https://verifier.example.com",
                    c.baseURL)
                assert.Equal(t, "test-key-123", c.apiKey)
                assert.Equal(t, 10*time.Second,
                    c.httpClient.Timeout)
            },
        },
        {
            name: "valid config with default timeout",
            cfg: &Config{
                APIURL:  "https://verifier.example.com",
                APIKey:  "test-key-456",
            },
            validate: func(t *testing.T, c *Client) {
                assert.Equal(t, 30*time.Second,
                    c.httpClient.Timeout)
            },
        },
    }
}
```

```

    },
    {
        name: "empty API URL",
        cfg: &Config{APIKey: "test-key"},
        wantErr: true,
        errContains: "APIURL is required",
    },
    {
        name: "empty API key",
        cfg: &Config{APIURL: "https://
verifier.example.com"},
        wantErr: true,
        errContains: "APIKey is required",
    },
    {
        name: "invalid URL scheme",
        cfg: &Config{APIURL: "ftp://
verifier.example.com", APIKey: "key"},
        wantErr: true,
        errContains: "URL scheme must be http or https",
    },
    {
        name: "timeout below minimum",
        cfg: &Config{APIURL: "https://v.example.com",
APIKey: "k", Timeout: 100 * time.Millisecond},
        wantErr: true,
        errContains: "timeout must be at least 1s",
    },
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        client, err := NewVerifierClient(tt.cfg)
        if tt.wantErr {
            require.Error(t, err)
            assert.Contains(t, err.Error(), tt.errContains)
            return
        }
        require.NoError(t, err)
        require.NotNil(t, client)
        tt.validate(t, client)
    })
}

func TestVerifierClient_Ping(t *testing.T) {
    t.Run("healthy server returns nil", func(t *testing.T) {
        server := httptest.NewServer(http.HandlerFunc(
            func(w http.ResponseWriter, r *http.Request) {
                assert.Equal(t, "/health", r.URL.Path)
                assert.Equal(t, "Bearer test-key",
r.Header.Get("Authorization"))
            }
        ))
    })
}

```

```

        w.WriteHeader(http.StatusOK)
        w.Write([]byte(`{"status":"healthy"}`))
    },
))
defer server.Close()

client, err := NewVerifierClient(&Config{APIURL:
server.URL, APIKey: "test-key"})
require.NoError(t, err)
assert.NoError(t, client.Ping(context.Background()))
})

t.Run("unhealthy server returns error", func(t *testing.T) {
    server := httptest.NewServer(http.HandlerFunc(
        func(w http.ResponseWriter, r *http.Request) {
            w.WriteHeader(http.StatusServiceUnavailable)
            w.Write([]byte(`{"status":"unhealthy"}`))
        },
    ))
    defer server.Close()

    client, _ := NewVerifierClient(&Config{APIURL:
server.URL, APIKey: "k"})
    err := client.Ping(context.Background())
    require.Error(t, err)
    assert.Contains(t, err.Error(), "unhealthy")
})

t.Run("timeout returns deadline exceeded", func(t *testing.T) {
    {
        server := httptest.NewServer(http.HandlerFunc(
            func(w http.ResponseWriter, r *http.Request) {
                time.Sleep(2 * time.Second)
                w.WriteHeader(http.StatusOK)
            },
        ))
        defer server.Close()

        client, _ := NewVerifierClient(&Config{APIURL:
server.URL, APIKey: "k", Timeout: 500 * time.Millisecond})
        ctx, cancel := context.WithTimeout(context.Background(),
300*time.Millisecond)
        defer cancel()

        err := client.Ping(ctx)
        require.Error(t, err)
        assert.True(t, errors.Is(err, context.DeadlineExceeded)
|| strings.Contains(err.Error(), "timeout"),
"expected timeout error, got: %v", err)
    }
})
}

```

The constructor tests validate six scenarios: valid config with explicit and default timeouts, empty API URL, empty API key, invalid URL scheme, and timeout below minimum. The Ping tests verify correct Authorization header handling, HTTP 200/503 responses, and timeout error propagation with `context.DeadlineExceeded` wrapping. The anti-bluff assertion on the timeout test checks for the specific error type, ensuring a DNS failure does not spuriously pass as a timeout.

8.2.2 Verification Pipeline Test Implementation

The verification pipeline in `internal/verifier/pipeline.go` orchestrates eight ordered steps: existence check, connectivity test, authentication validation, completion verification, capability detection, translation quality assessment, latency benchmarking, and error handling classification. The test file `internal/verifier/pipeline_test.go` tests each step with injected pass/fail outcomes, validates step ordering, verifies error propagation from critical steps, and tests retry logic.

Code Block 8.2: `internal/verifier/pipeline_test.go`

```
package verifier

import (
    "context"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

type mockStep struct {
    name          string
    shouldFail    bool
    isCritical    bool
    called        bool
    duration      time.Duration
}

func (m *mockStep) Execute(ctx context.Context, model
    *ModelContext) StepResult {
    m.called = true
    time.Sleep(m.duration)
    if m.shouldFail {
        return StepResult{StepName: m.name, Passed: false,
            Critical: m.isCritical, Error: errors.New(m.name + "
            failed")}
    }
    return StepResult{StepName: m.name, Passed: true, Critical:
        m.isCritical}
}

type orderRecordingStep struct {
    wrapped PipelineStep
```

```

    order    *[]string
}

func (o *orderRecordingStep) Execute(ctx context.Context, model
    *ModelContext) StepResult {
    *o.order = append(*o.order, o.wrapped.(*mockStep).name)
    return o.wrapped.Execute(ctx, model)
}

func TestPipeline_StepOrdering(t *testing.T) {
    var executionOrder []string
    steps := []PipelineStep{
        &mockStep{name: "existence", duration: 1 *
            time.Millisecond},
        &mockStep{name: "connectivity", duration: 1 *
            time.Millisecond},
        &mockStep{name: "authentication", duration: 1 *
            time.Millisecond},
        &mockStep{name: "completion", duration: 1 *
            time.Millisecond},
        &mockStep{name: "capability", duration: 1 *
            time.Millisecond},
        &mockStep{name: "quality", duration: 1 *
            time.Millisecond},
        &mockStep{name: "latency", duration: 1 *
            time.Millisecond},
        &mockStep{name: "error_handling", duration: 1 *
            time.Millisecond},
    }
    for i := range steps {
        steps[i] = &orderRecordingStep{wrapped: steps[i], order:
            &executionOrder}
    }

    pipeline := NewPipeline(steps)
    result := pipeline.Run(context.Background(),
        &ModelContext{ModelID: "gpt-4", ProviderID: "openai"})

    require.NoError(t, result.Error)
    require.Equal(t, 8, len(executionOrder))
    assert.Equal(t, "existence", executionOrder[0])
    assert.Equal(t, "connectivity", executionOrder[1])
    assert.Equal(t, "authentication", executionOrder[2])
    assert.Equal(t, "latency", executionOrder[6])
    assert.Equal(t, "error_handling", executionOrder[7])
}

func TestPipeline_ErrorPropagation(t *testing.T) {
    tests := []struct {
        name            string
        failedStep      string
        isCritical       bool
        expectError      bool
    }{

```



```

{name: "critical existence failure aborts", failedStep:
"existence", isCritical: true, expectError: true},
{name: "critical auth failure aborts", failedStep:
"authentication", isCritical: true, expectError: true},
{name: "non-critical quality failure continues",
failedStep: "quality", isCritical: false, expectError:
false},
{name: "non-critical latency failure continues",
failedStep: "latency", isCritical: false, expectError:
false},
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        steps := []PipelineStep{
            &mockStep{name: "existence", shouldFail:
tt.failedStep == "existence", isCritical: true},
            &mockStep{name: "connectivity", shouldFail:
tt.failedStep == "connectivity", isCritical: true},
            &mockStep{name: "authentication", shouldFail:
tt.failedStep == "authentication", isCritical: true},
            &mockStep{name: "completion", shouldFail:
tt.failedStep == "completion", isCritical: false},
            &mockStep{name: "capability", shouldFail:
tt.failedStep == "capability", isCritical: false},
            &mockStep{name: "quality", shouldFail:
tt.failedStep == "quality", isCritical: false},
            &mockStep{name: "latency", shouldFail:
tt.failedStep == "latency", isCritical: false},
            &mockStep{name: "error_handling", shouldFail:
tt.failedStep == "error_handling", isCritical: true},
        }

        pipeline := NewPipeline(steps)
        result := pipeline.Run(context.Background(),
&ModelContext{ModelID: "gpt-4", ProviderID: "openai"})

        if tt.expectError {
            require.Error(t, result.Error)
            assert.Contains(t, result.Error.Error(),
tt.failedStep)
            for _, s := range steps {
                m := s.(*mockStep)
                if m.name != tt.failedStep && m.isCritical {
                    assert.False(t, m.called, "critical step
%s should not run after abort", m.name)
                }
            }
        } else {
            require.NoError(t, result.Error)
            for _, s := range steps {
                assert.True(t, s.(*mockStep).called, "step %s
should have been called", s.(*mockStep).name)
            }
        }
    })
}

```

```

    })
}

type flakyMockStep struct {
    attempts int
    failCount int
}

func (f *flakyMockStep) Execute(ctx context.Context, model
    *ModelContext) StepResult {
    f.attempts++
    if f.attempts <= f.failCount {
        return StepResult{StepName: "flaky", Passed: false,
            Critical: true, Error: errors.New("transient failure")}
    }
    return StepResult{StepName: "flaky", Passed: true, Critical:
        true}
}

func TestPipeline_RetryLogic(t *testing.T) {
    flaky := &flakyMockStep{failCount: 2}
    steps := []PipelineStep{flaky, &mockStep{name: "completion"}}

    pipeline := NewPipelineWithRetry(steps,
        RetryConfig{MaxAttempts: 3, Backoff: 10 *
            time.Millisecond})
    result := pipeline.Run(context.Background(),
        &ModelContext{ModelID: "gpt-4"})

    require.NoError(t, result.Error)
    assert.Equal(t, 3, flaky.attempts, "expected 3 attempts (2
        failures + 1 success)")
}

```

The mockStep struct implements PipelineStep and records execution for post-test assertions. TestPipeline_StepOrdering wraps each step in an orderRecordingStep and asserts the eight-step sequence, catching any refactoring that reorders steps. TestPipeline_ErrorPropagation covers four scenarios: critical failures that abort the pipeline and non-critical failures that allow continuation. The anti-bluff check verifies that for critical failures subsequent steps were not called, and for non-critical failures all eight steps executed. The retry test uses a flakyMockStep that fails twice before succeeding; the assertion checks attempts == 3, verifying the retry mechanism actually re-invoked the step.

8.3 Unit Testing — Scoring and Adapter Components

8.3.1 Scoring Engine Test Implementation

The scoring engine in `internal/verifier/scoring/engine.go` computes five component scores and aggregates them into a weighted total. The test file `internal/verifier/scoring/engine_test.go` contains 20+ test cases with known inputs and mathematically expected outputs.

Code Block 8.3: `internal/verifier/scoring/engine_test.go`

```
package scoring

import (
    "context"
    "testing"
    "time"

    "digital.vasic.llmsverifier/database"
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

type ScoreTestCase struct {
    name                string
    modelData           *ModelData
    dbModel             *database.Model
    expectedSpeed        float64
    expectedEfficiency   float64
    expectedCost         float64
    expectedCapability   float64
    expectedRecency      float64
    expectedOverall      float64
    tolerance            float64
}

func TestCalculateComprehensiveScore_KnownInputs(t *testing.T) {
    recent := time.Date(2024, 6, 1, 0, 0, 0, 0, time.UTC)
    old := time.Date(2022, 1, 1, 0, 0, 0, 0, time.UTC)
    ctxTokens := 128000
    paramSmall := int64(500_000_000)
    paramLarge := int64(175_000_000_000)

    tests := []ScoreTestCase{
        {
            name: "optimal model – high across all components",
            modelData: &ModelData{
                ID: "gpt-4-optimal", Name: "GPT-4 Optimal",
                Provider: "OpenAI",
                ContextWindow: 128000, InputTokenCost: 1.0,
                ThroughputRPS: 15.0,
            },
        },
    }
}
```

```

        LatencyMs: 200, ParameterCount: paramSmall,
        OpenSource: false, Multimodal: true, Reasoning:
true, LastUpdated: recent,
    },
    dbModel: &database.Model{
        ModelID: "gpt-4-optimal", ResponsivenessScore:
9.0,
        ParameterCount: &paramSmall, ContextWindowTokens:
&ctxTokens,
        IsMultimodal: true, SupportsReasoning: true,
        ReleaseDate: &recent, TrainingDataCutoff:
&recent, LastVerified: &recent,
    },
    expectedSpeed: 9.0, expectedEfficiency: 10.0,
    expectedCost: 9.0,
    expectedCapability: 8.5, expectedRecency: 10.0,
    expectedOverall: 9.2,
    tolerance: 0.1,
},
{
    name: "expensive large model – low cost, high
capability",
    modelData: &ModelData{
        ID: "gpt-4-expensive", Name: "GPT-4 Expensive",
Provider: "OpenAI",
        ContextWindow: 128000, InputTokenCost: 30.0,
ThroughputRPS: 5.0,
        LatencyMs: 2000, ParameterCount: paramLarge,
        OpenSource: false, Multimodal: true, Reasoning:
true, LastUpdated: recent,
    },
    dbModel: &database.Model{
        ModelID: "gpt-4-expensive", ResponsivenessScore:
5.0,
        ParameterCount: &paramLarge, ContextWindowTokens:
&ctxTokens,
        IsMultimodal: true, SupportsReasoning: true,
        ReleaseDate: &recent, TrainingDataCutoff:
&recent, LastVerified: &recent,
    },
    expectedSpeed: 6.0, expectedEfficiency: 8.0,
    expectedCost: 2.0,
    expectedCapability: 8.5, expectedRecency: 10.0,
    expectedOverall: 6.2,
    tolerance: 0.1,
},
{
    name: "old cheap model – high cost, low recency",
    modelData: &ModelData{
        ID: "old-cheap", Name: "Old Cheap", Provider:
"OpenSource",
        ContextWindow: 32000, InputTokenCost: 0.1,
ThroughputRPS: 3.0,
        LatencyMs: 3000, ParameterCount: paramSmall,

```

```

        OpenSource: true, Multimodal: false, Reasoning:
false, LastUpdated: old,
    },
    dbModel: &database.Model{
        ModelID: "old-cheap", ResponsivenessScore: 4.0,
        ParameterCount: &paramSmall, ContextWindowTokens:
nil,
        IsMultimodal: false, SupportsReasoning: false,
        ReleaseDate: &old, TrainingDataCutoff: &old,
        LastVerified: &old,
        OpenSource: true,
    },
    expectedSpeed: 4.0, expectedEfficiency: 7.0,
    expectedCost: 10.0,
    expectedCapability: 5.0, expectedRecency: 2.0,
    expectedOverall: 6.0,
    tolerance: 0.1,
},
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        db := setupTestDB(t)
        defer cleanupTestDB(t, db)

        provider := &database.Provider{Name: "TestProvider",
Endpoint: "https://api.test.com", IsActive: true}
        require.NoError(t, db.CreateProvider(provider))
        tt.dbModel.ProviderID = provider.ID
        require.NoError(t, db.CreateModel(tt.dbModel))

        mockClient := NewMockModelsDevClient()
        mockClient.AddMockModel(ModelsDevModel{
            ModelID: tt.modelData.ID, Model:
tt.modelData.Name,
            Provider: tt.modelData.Provider, InputCostPer1M:
tt.modelData.InputTokenCost,
            ContextLimit: tt.modelData.ContextWindow,
            ReleaseDate:
tt.modelData.LastUpdated.Format("2006-01-02"),
            AdditionalData: ModelsDevAdditionalData{
                ParameterCount: tt.modelData.ParameterCount,
                OpenWeights:      tt.modelData.OpenSource,
                Multimodal:      tt.modelData.Multimodal,
            },
        })

        engine := NewScoringEngine(db, mockClient,
&logging.Logger{})
        score, err :=
engine.CalculateComprehensiveScore(context.Background(),
tt.modelData.ID,
            ScoringConfig{Weights: DefaultScoreWeights()})
        require.NoError(t, err)
    })
}

```

```

        require.NotNil(t, score)

        assert.InDelta(t, tt.expectedSpeed,
            score.Components.SpeedScore, tt.tolerance)
        assert.InDelta(t, tt.expectedEfficiency,
            score.Components.EfficiencyScore, tt.tolerance)
        assert.InDelta(t, tt.expectedCost,
            score.Components.CostScore, tt.tolerance)
        assert.InDelta(t, tt.expectedCapability,
            score.Components.CapabilityScore, tt.tolerance)
        assert.InDelta(t, tt.expectedRecency,
            score.Components.RecencyScore, tt.tolerance)
        assert.InDelta(t, tt.expectedOverall,
            score.OverallScore, tt.tolerance)

        // Anti-bluff: verify persistence
        retrieved, err := db.GetModel(tt.dbModel.ID)
        require.NoError(t, err)
        assert.InDelta(t, tt.expectedOverall,
            retrieved.OverallScore, tt.tolerance)
    })
}

func TestCalculateComprehensiveScore_BoundaryConditions(t
    *testing.T) {
    tests := []struct {
        name          string
        setupDBModel func() *database.Model
    }{
        {
            name: "zero parameter count",
            setupDBModel: func() *database.Model {
                z := int64(0)
                return &database.Model{ParameterCount: &z,
                    OpenSource: false}
            },
        },
        {
            name: "nil release date",
            setupDBModel: func() *database.Model {
                return &database.Model{ReleaseDate: nil,
                    LastVerified: nil}
            },
        },
        {
            name: "all nil optional fields",
            setupDBModel: func() *database.Model {
                return &database.Model{
                    ParameterCount: nil, ContextWindowTokens:
                    nil, ReleaseDate: nil,
                    TrainingDataCutoff: nil, LastVerified: nil,
                    IsMultimodal: false,
                    SupportsReasoning: false, OpenSource: false,

```

```

    },
    },
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        db := setupTestDB(t)
        defer cleanupTestDB(t, db)

        provider := &database.Provider{Name: "Test",
Endpoint: "https://test.com", IsActive: true}
        require.NoError(t, db.CreateProvider(provider))
        dbModel := tt.setupDBModel()
        dbModel.ProviderID = provider.ID
        dbModel.ModelID = "boundary-test"
        dbModel.Name = "Boundary Test"
        require.NoError(t, db.CreateModel(dbModel))

        mockClient := NewMockModelsDevClient()
        mockClient.AddMockModel(models.DevModel{ModelID:
"boundary-test", InputCostPer1M: 5.0, ContextLimit: 8000})

        engine := NewScoringEngine(db, mockClient,
&logging.Logger{})
        score, err :=
engine.CalculateComprehensiveScore(context.Background(),
"boundary-test",
        ScoringConfig{Weights: DefaultScoreWeights()})

        require.NoError(t, err)
        assert.GreaterOrEqual(t, score.OverallScore, 0.0)
        assert.LessOrEqual(t, score.OverallScore, 10.0)
        assert.GreaterOrEqual(t, score.Components.SpeedScore,
0.0)
        assert.LessOrEqual(t, score.Components.SpeedScore,
10.0)
    })
}
}

```

The test matrix covers three archetypes: optimal (high all components), expensive large (low cost, high capability), and old cheap open-source (high cost, low recency). Each case pre-computes the expected overall using the weight formula and asserts within 0.1 tolerance. The anti-bluff check queries the database after scoring to verify persistence. Boundary conditions cover zero parameter count, nil release date, and all-nil optional fields — asserting the score always falls within [0, 10].

8.3.2 Score Adapter Test Implementation

The score adapter in `internal/services/llmsverifier_score_adapter.go` converts `ComprehensiveScore` to `ProviderScore`. The test file validates field-by-field mapping, cache behavior, and error handling.

Code Block 8.4: internal/services/llmsverifier_score_adapter_test.go

```
package services

import (
    "context"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestScoreAdapter_FieldMapping(t *testing.T) {
    tests := []struct {
        name          string
        input          *llmverifier.ComprehensiveScore
        expectedOutput *ProviderScore
    }{
        {
            name: "complete score with all components",
            input: &llmverifier.ComprehensiveScore{
                ModelID: "gpt-4", ModelName: "GPT-4 (SC:8.5)",
                OverallScore: 8.5,
                Components: llmverifier.ScoreComponents{
                    SpeedScore: 9.0, EfficiencyScore: 8.0,
                    CostScore: 7.5,
                    CapabilityScore: 8.5, RecencyScore: 9.0,
                },
                LastCalculated: time.Date(2024, 6, 15, 12, 0, 0,
                    0, time.UTC),
                DataSource: "models.dev",
            },
            expectedOutput: &ProviderScore{
                ProviderID: "gpt-4", ProviderName: "GPT-4",
                OverallScore: 8.5,
                SpeedScore: 9.0, EfficiencyScore: 8.0, CostScore:
                    7.5,
                CapabilityScore: 8.5, RecencyScore: 9.0,
                ScoreSuffix: "(SC:8.5)", ScoreSource:
                    "llmsverifier",
                LastUpdated: time.Date(2024, 6, 15, 12, 0, 0, 0,
                    time.UTC),
            },
        },
        {
            name: "suffix extraction from model name",
            input: &llmverifier.ComprehensiveScore{
                ModelID: "claude-3-opus", ModelName: "Claude 3
                Opus (SC:9.2)", OverallScore: 9.2,
                Components: llmverifier.ScoreComponents{
                    SpeedScore: 8.5, EfficiencyScore: 9.0,
                    CostScore: 6.0,
                    CapabilityScore: 9.5, RecencyScore: 8.0,

```



```

        },
        expectedOutput: &ProviderScore{
            ProviderID: "claude-3-opus", ProviderName:
"Claude 3 Opus",
            OverallScore: 9.2, SpeedScore: 8.5,
            EfficiencyScore: 9.0,
            CostScore: 6.0, CapabilityScore: 9.5,
            RecencyScore: 8.0,
            ScoreSuffix: "(SC:9.2)",
        },
    },
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        adapter := NewScoreAdapter(&mockVerifierClient{
            scores:
map[string]*llmverifier.ComprehensiveScore{tt.input.ModelID:
tt.input},
        })

        result, err := adapter.GetScore(context.Background(),
tt.input.ModelID)
        require.NoError(t, err)

        assert.Equal(t, tt.expectedOutput.ProviderID,
result.ProviderID)
        assert.Equal(t, tt.expectedOutput.ProviderName,
result.ProviderName)
        assert.InDelta(t, tt.expectedOutput.OverallScore,
result.OverallScore, 0.01)
        assert.InDelta(t, tt.expectedOutput.SpeedScore,
result.SpeedScore, 0.01)
        assert.InDelta(t, tt.expectedOutput.EfficiencyScore,
result.EfficiencyScore, 0.01)
        assert.InDelta(t, tt.expectedOutput.CostScore,
result.CostScore, 0.01)
        assert.InDelta(t, tt.expectedOutput.CapabilityScore,
result.CapabilityScore, 0.01)
        assert.InDelta(t, tt.expectedOutput.RecencyScore,
result.RecencyScore, 0.01)
        assert.Equal(t, tt.expectedOutput.ScoreSuffix,
result.ScoreSuffix)
        assert.Equal(t, tt.expectedOutput.ScoreSource,
result.ScoreSource)
    })
}

}

func TestScoreAdapter_CacheBehavior(t *testing.T) {
    callCount := 0
    mockClient := &countingVerifierClient{
        getScoreFunc: func(ctx context.Context, modelID string)
(*llmverifier.ComprehensiveScore, error) {

```

```

        callCount++
        return &llmverifier.ComprehensiveScore{ModelID:
modelID, OverallScore: 7.5, LastCalculated: time.Now()},
        nil
    },
}

adapter := NewScoreAdapter(mockClient)
adapter.SetCacheTTL(5 * time.Minute)
ctx := context.Background()

score1, err := adapter.GetScore(ctx, "gpt-4")
require.NoError(t, err)
assert.Equal(t, 1, callCount, "first call should hit
    verifier")

score2, err := adapter.GetScore(ctx, "gpt-4")
require.NoError(t, err)
assert.Equal(t, 1, callCount, "second call should use cache")

// Anti-bluff: verify cached score is a copy
score1.OverallScore = 99.9
assert.Equal(t, 7.5, score2.OverallScore, "mutating result
    should not affect cache")

adapter.InvalidateCache("gpt-4")
_, err = adapter.GetScore(ctx, "gpt-4")
require.NoError(t, err)
assert.Equal(t, 2, callCount, "after invalidation, should re-
    fetch")
}

func TestScoreAdapter_ErrorHandling(t *testing.T) {
    t.Run("verifier unreachable returns fallback", func(t
        *testing.T) {
        mockClient := &failingVerifierClient{err:
            context.DeadlineExceeded}
        adapter := NewScoreAdapter(mockClient)

        score, err := adapter.GetScore(context.Background(),
            "gpt-4")
        require.NoError(t, err, "adapter should return fallback,
            not error")
        require.NotNil(t, score)
        assert.InDelta(t, 5.0, score.OverallScore, 0.01)
        assert.Equal(t, "fallback", score.ScoreSource)
    })

    t.Run("not-found error propagates", func(t *testing.T) {
        mockClient := &failingVerifierClient{err:
            llmverifier.ErrModelNotFound}
        adapter := NewScoreAdapter(mockClient)

```

```

        _, err := adapter.GetScore(context.Background(), "unknown-
        model")
        require.Error(t, err)
        assert.ErrorIs(t, err, ErrModelNotInRegistry)
    })
}

```

The field mapping test compares all ten fields individually — a struct equality check would miss a field swap. The cache test uses a `CountingVerifierClient` and anti-bluff checks that mutating the returned pointer does not affect the cached copy. Error handling covers two modes: unreachable verifier returns a neutral fallback score of 5.0, and model-not-found propagates as `ErrModelNotInRegistry`.

8.4 Unit Testing — Discovery and Registry

8.4.1 Discovery Service Tests

The discovery service in `internal/verifier/discovery/service.go` implements 3-tier fallback: static registry, provider API, models.dev. `internal/verifier/discovery/service_test.go` mocks all three tiers to inject deterministic failure modes, tests that tier-1 failure triggers tier-2 within the 5-second timeout, and verifies deduplication prevents duplicate models when found in multiple tiers.

8.4.2 Registry Tests

The registry in `internal/verifier/discovery/registry.go` provides CRUD and search with filtering. `internal/verifier/discovery/registry_test.go` uses a real SQLite `:memory:` database — not a mock — because SQL queries contain `LIKE`, `BETWEEN`, and `JOIN` clauses that mocks would silently pass with invalid SQL. Tests cover CRUD, search with `provider/score/capability` filters, `FilterByScore` boundary conditions, and `FilterByProvider` with invalid provider ID.

8.4.3 Gatekeeper Tests

The gatekeeper in `internal/verifier/discovery/gatekeeper.go` determines model availability from three inputs: verification status (2 values), score status (2 values), and health status (3 values) — 12 combinations. `internal/verifier/discovery/gatekeeper_test.go` covers all 12 plus boundary conditions.

Code Block 8.5: `internal/verifier/discovery/gatekeeper_test.go`

```

package discovery

import (
    "testing"
    "time"

    "github.com/stretchr/testify/assert"

```

)

```
type GatekeeperDecision struct {
    name                string
    verified             bool
    scoreAboveThreshold bool
    healthStatus        string
    expectedAvailable   bool
    expectedReason      string
}

func TestGatekeeper_DecisionMatrix(t *testing.T) {
    gk := NewGatekeeper(GatekeeperConfig{MinScoreThreshold: 6.0})

    tests := []GatekeeperDecision{
        {verified: true, scoreAboveThreshold: true, healthStatus:
        "healthy", expectedAvailable: true, expectedReason: ""},
        {verified: true, scoreAboveThreshold: true, healthStatus:
        "degraded", expectedAvailable: true, expectedReason: ""},
        {verified: true, scoreAboveThreshold: true, healthStatus:
        "unavailable", expectedAvailable: false, expectedReason:
        "health"},
        {verified: true, scoreAboveThreshold: false,
        healthStatus: "healthy", expectedAvailable: false,
        expectedReason: "score"},
        {verified: true, scoreAboveThreshold: false,
        healthStatus: "degraded", expectedAvailable: false,
        expectedReason: "score"},
        {verified: true, scoreAboveThreshold: false,
        healthStatus: "unavailable", expectedAvailable: false,
        expectedReason: "score"},
        {verified: false, scoreAboveThreshold: true,
        healthStatus: "healthy", expectedAvailable: false,
        expectedReason: "verification"},
        {verified: false, scoreAboveThreshold: true,
        healthStatus: "degraded", expectedAvailable: false,
        expectedReason: "verification"},
        {verified: false, scoreAboveThreshold: true,
        healthStatus: "unavailable", expectedAvailable: false,
        expectedReason: "verification"},
        {verified: false, scoreAboveThreshold: false,
        healthStatus: "healthy", expectedAvailable: false,
        expectedReason: "verification"},
        {verified: false, scoreAboveThreshold: false,
        healthStatus: "degraded", expectedAvailable: false,
        expectedReason: "verification"},
        {verified: false, scoreAboveThreshold: false,
        healthStatus: "unavailable", expectedAvailable: false,
        expectedReason: "verification"},
    }

    for i, tt := range tests {
        t.Run(fmt.Sprintf("case_%d_v%v_s%v_h%s", i, tt.verified,
        tt.scoreAboveThreshold, tt.healthStatus), func(t
        *testing.T) {
```

```

        model := &RegistryModel{
            ModelID:      "test-model",
            Verified:      tt.verified,
            OverallScore:
gatekeeperTestScore(tt.scoreAboveThreshold),
            HealthStatus: tt.healthStatus,
            LastVerified:
gatekeeperTestTimestamp(tt.verified),
        }

        available, reason := gk.IsAvailable(model)
        assert.Equal(t, tt.expectedAvailable, available,
            "availability mismatch for v=%v s=%v h=%s",
            tt.verified, tt.scoreAboveThreshold, tt.healthStatus)
        assert.Equal(t, tt.expectedReason, reason,
            "reason mismatch for v=%v s=%v h=%s",
            tt.verified, tt.scoreAboveThreshold, tt.healthStatus)
    })
}

}

func TestGatekeeper_BoundaryConditions(t *testing.T) {
    gk := NewGatekeeper(GatekeeperConfig{MinScoreThreshold: 6.0})

    t.Run("exactly at threshold", func(t *testing.T) {
        model := &RegistryModel{ModelID: "boundary", Verified:
            true, OverallScore: 6.0, HealthStatus: "healthy"}
        available, _ := gk.IsAvailable(model)
        assert.True(t, available,
            "model at exact threshold should be available")
    })

    t.Run("one epsilon below", func(t *testing.T) {
        model := &RegistryModel{ModelID: "boundary", Verified:
            true, OverallScore: 5.999, HealthStatus: "healthy"}
        available, reason := gk.IsAvailable(model)
        assert.False(t, available)
        assert.Equal(t, "score", reason)
    })

    t.Run("negative score", func(t *testing.T) {
        model := &RegistryModel{ModelID: "negative", Verified:
            true, OverallScore: -1.0, HealthStatus: "healthy"}
        available, reason := gk.IsAvailable(model)
        assert.False(t, available)
        assert.Equal(t, "score", reason)
    })
}

func gatekeeperTestScore(above bool) float64 {
    if above {
        return 8.5
    }
    return 4.0
}

```

```

}

func gatekeeperTestTimestamp(verified bool) *time.Time {
    if !verified {
        return nil
    }
    ts := time.Date(2024, 6, 1, 0, 0, 0, 0, time.UTC)
    return &ts
}

```

The decision matrix enumerates all 12 combinations. The anti-bluff check asserts both the availability boolean and the rejection reason — a boolean-only check could pass even with an incorrect reason that would mislead operators during incidents. Boundary conditions cover the exact threshold (6.0), one epsilon below (5.999), and a negative score (-1.0) to ensure the comparison operator is $\geq$ not $>$.

8.5 Integration Testing — End-to-End Verification Flow

8.5.1 Integration Test Environment and Full User Journey

The integration test `tests/integration/verifier_flow_test.go` validates the complete verification pipeline using a dockerized LLMsVerifier via Docker Compose. The `test-compose.yml` specifies three services: LLMsVerifier on port 18080, mock providers via `mockserver/mockserver` on port 11080, and HelixTranslate depending on both with `condition: service_healthy`.

Code Block 8.6: `tests/integration/verifier_flow_test.go`

```

package integration

import (
    "context"
    "fmt"
    "net/http"
    "os"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

const TestComposeFile = "../test-compose.yml"

func TestVerifierFlow_FullUserJourney(t *testing.T) {
    if os.Getenv("RUN_INTEGRATION_TESTS") == "" {
        t.Skip("set RUN_INTEGRATION_TESTS=1 to run integration tests")
    }
}

```

```

ctx := context.Background()
compose := testcompose.Up(t, TestComposeFile,
    testcompose.WithTimeout(2*time.Minute),
    testcompose.WaitForService("verifier", "/health",
        http.StatusOK),
)
defer compose.Down()

verifierURL := compose.ServiceURL("verifier")

// Step 1: Configure mock provider
configureMockProviders(t, compose.ServiceURL("mock-
    providers"))

// Step 2: Trigger discovery
discoverResp := mustPost(t, verifierURL+"/api/v1/models/
    discover", map[string]any{
        "provider": "mock-openai",
    })
require.Equal(t, "discovery_started", discoverResp["status"])

// Step 3: Poll for discovery
var discoveredModels []map[string]any
require.Eventually(t, func() bool {
    models := mustGet(t, verifierURL+"/api/v1/models")
    discoveredModels = models["models"].([]map[string]any)
    return len(discoveredModels) > 0
}, 60*time.Second, 2*time.Second, "discovery should find
    models")

// Step 4: Verify first discovered model
modelID := fmt.Sprintf("%.0f", discoveredModels[0]["id"].
    (float64))
verifyResp := mustPost(t, verifierURL+"/api/v1/
    models/"+modelID+"/verify", nil)
require.Equal(t, "verification_started", verifyResp["status"])

// Step 5: Poll for verification (8-step pipeline, max 120s)
var verifiedModel map[string]any
require.Eventually(t, func() bool {
    model := mustGet(t, verifierURL+"/api/v1/models/"+modelID)
    status, ok := model["status"].(string)
    verifiedModel = model
    return ok && status == "verified"
}, 120*time.Second, 5*time.Second, "verification should
    complete")

// Step 6: Anti-bluff – score in valid range
score, ok := verifiedModel["score"].(float64)
require.True(t, ok, "verified model should have score")
assert.GreaterOrEqual(t, score, 0.0)
assert.LessOrEqual(t, score, 10.0)

```

```

// Step 7: Select model for translation
selectResp := mustPost(t, verifierURL+"/api/v1/models/
    select", map[string]any{
        "task": "translation", "source_lang": "en",
        "target_lang": "de",
    })
selectedID := selectResp["selected_model_id"].(string)

// Anti-bluff: selected model must be from discovered set
found := false
for _, m := range discoveredModels {
    if fmt.Sprintf("%.0f", m["id"].(float64)) == selectedID {
        found = true
        break
    }
}
assert.True(t, found, "selected model must come from
    discovered set")
}

func TestVerifierFlow_ModelDegradation(t *testing.T) {
    if os.Getenv("RUN_INTEGRATION_TESTS") == "" {
        t.Skip("set RUN_INTEGRATION_TESTS=1")
    }

    compose := testcompose.Up(t, TestComposeFile)
    defer compose.Down()

    verifierURL := compose.ServiceURL("verifier")

    // Seed model with high score
    mustPost(t, verifierURL+"/api/v1/models", map[string]any{
        "model_id": "degradable-model", "name": "Degradable
        (SC:8.5)",
        "provider": "mock-openai", "score": 8.5, "status":
        "verified",
    })

    // Drop score below threshold
    mustPut(t, verifierURL+"/api/v1/models/degradable-model",
        map[string]any{"score": 3.5})

    // Select should NOT pick degraded model
    selectResp, _ := httpPost(verifierURL+"/api/v1/models/
        select", map[string]any{
            "task": "translation", "source_lang": "en",
            "target_lang": "de",
        })
    selectedID, ok := selectResp["selected_model_id"].(string)
    if ok && selectedID == "degradable-model" {
        t.Fatal("degraded model with score 3.5 was selected -
            gatekeeper failed")
    }
}

```


The full journey exercises seven steps: configure mock, trigger discovery, poll for completion, verify model, poll for verification, assert score in [0, 10], select for translation, and anti-bluff check that the selection came from the discovered set. The degradation test simulates a score drop from 8.5 to 3.5 and asserts the selector avoids the degraded model — the exact failure mode the gatekeeper exists to prevent.

8.6 Integration Testing — API Endpoints

8.6.1 API Integration Test Implementation

tests/integration/api_models_test.go exercises all /api/v1/models/* endpoints with JWT auth, pagination, filtering, sorting, and concurrent access. The setup seeds 50 models across 5 providers before each run.

Code Block 8.7: tests/integration/api_models_test.go

```
package integration

import (
    "fmt"
    "net/http"
    "sync"
    "testing"
    "time"

    "github.com/golang-jwt/jwt/v5"
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

type AuthenticatedTestClient struct {
    baseURL    string
    httpClient *http.Client
    token      string
}

func NewAuthenticatedTestClient(baseURL, secret string)
    *AuthenticatedTestClient {
    token, _ := jwt.NewWithClaims(jwt.SigningMethodHS256,
        jwt.MapClaims{
            "sub": "test-user", "role": "admin",
            "iat": time.Now().Unix(), "exp":
                time.Now().Add(time.Hour).Unix(),
        }).SignedString([]byte(secret))
    return &AuthenticatedTestClient{
        baseURL: baseURL, httpClient: &http.Client{Timeout: 10 *
            time.Second}, token: token,
    }
}
```

```

func (c *AuthenticatedTestClient) Get(path string)
    (map[string]any, error) {
    req, _ := http.NewRequest("GET", c.baseURL+path, nil)
    req.Header.Set("Authorization", "Bearer "+c.token)
    resp, err := c.httpClient.Do(req)
    if err != nil {
        return nil, err
    }
    defer resp.Body.Close()
    var result map[string]any
    json.NewDecoder(resp.Body).Decode(&result)
    return result, nil
}

func SeedTestDatabase(t *testing.T, verifierURL string) {
    client := NewAuthenticatedTestClient(verifierURL, "test-
        secret")
    providerIDs := make([]string, 5)
    for i := 0; i < 5; i++ {
        resp, _ := client.Post("/api/v1/providers",
            map[string]any{
                "name": fmt.Sprintf("Provider-%d", i), "endpoint":
                fmt.Sprintf("https://api.p%d.com", i),
                "is_active": i < 4,
            })
        providerIDs[i] = resp["id"].(string)
    }
    for i := 0; i < 50; i++ {
        score := float64(i%10) + 1.0
        client.Post("/api/v1/models", map[string]any{
            "model_id": fmt.Sprintf("model-%03d", i), "name":
            fmt.Sprintf("Model %d (SC:%.1f)", i, score),
            "provider": providerIDs[i%5], "score": score,
            "status": "verified",
            "capabilities": []string{"text", "translation"},
        })
    }
}

func TestAPI_ModelsPagination(t *testing.T) {
    compose := testcompose.Up(t, TestComposeFile)
    defer compose.Down()
    verifierURL := compose.ServiceURL("verifier")
    SeedTestDatabase(t, verifierURL)

    client := NewAuthenticatedTestClient(verifierURL, "test-
        secret")
    page1, _ := client.Get("/api/v1/models?limit=10&offset=0")
    models1 := page1["models"].([]any)
    assert.Len(t, models1, 10)
    assert.Equal(t, float64(50), page1["total"])

    page2, _ := client.Get("/api/v1/models?limit=10&offset=10")
    models2 := page2["models"].([]any)

```

```

assert.Len(t, models2, 10)

// Anti-bluff: pages should not overlap
page1IDs := extractModelIDs(t, models1)
page2IDs := extractModelIDs(t, models2)
for _, id := range page1IDs {
    assert.NotContains(t, page2IDs, id, "pages should not
        overlap")
}
}

func TestAPI_ModelsFiltering(t *testing.T) {
    compose := testcompose.Up(t, TestComposeFile)
    defer compose.Down()
    verifierURL := compose.ServiceURL("verifier")
    SeedTestDatabase(t, verifierURL)

    client := NewAuthenticatedTestClient(verifierURL, "test-
        secret")
    filtered, _ := client.Get("/api/v1/models?min_score=7.0")
    models := filtered["models"].([]any)
    assert.Greater(t, len(models), 0)
    for _, m := range models {
        assert.GreaterOrEqual(t, m.(map[string]any)["score"].
            (float64), 7.0)
    }
}

func TestAPI_ConcurrentAccess(t *testing.T) {
    compose := testcompose.Up(t, TestComposeFile)
    defer compose.Down()
    verifierURL := compose.ServiceURL("verifier")
    SeedTestDatabase(t, verifierURL)

    client := NewAuthenticatedTestClient(verifierURL, "test-
        secret")
    var wg sync.WaitGroup
    errors := make(chan error, 100)

    for i := 0; i < 100; i++ {
        wg.Add(1)
        go func(idx int) {
            defer wg.Done()
            resp, err := client.Get(fmt.Sprintf("/api/v1/models?
                limit=10&offset=%d", idx))
            if err != nil {
                errors <- err
                return
            }
            if len(resp["models"].([]any)) == 0 && idx < 50 {
                errors <- fmt.Errorf("request %d: expected
                    models", idx)
            }
        }(i)
    }
}

```

```

    }
    wg.Wait()
    close(errors)

    var errorList []error
    for e := range errors {
        errorList = append(errorList, e)
    }
    assert.Empty(t, errorList, "concurrent requests produced %d
        errors", len(errorList))
}

```

SeedTestDatabase creates 5 providers (4 active, 1 inactive) and 50 models with scores 1.0–10.0. The pagination test requests two pages and anti-bluff checks they do not overlap — a bug where offset is ignored would return identical pages. The filtering test asserts every returned model has score ≥ 7.0 . The concurrent access test fires 100 goroutines and collects errors through a channel, reporting specific failures rather than just a count.

8.7 Anti-Bluff Challenge Testing

8.7.1–8.7.4 Challenge Script Overview

Four challenge scripts validate subsystem behavior in production-like environments. `score_adapter_challenge.sh` compares adapter output field-by-field against `testdata/expected_scores.json`. `gatekeeping_challenge.sh` feeds all 12 matrix combinations to the gatekeeper CLI and compares JSON output. `discovery_resilience_challenge.sh` blocks each discovery tier via iptables and verifies fallback returns results. `selection_fairness_challenge.sh` creates 10 equivalent models, runs 1000 selections, and verifies no model exceeds 15% of selections.

8.7.5 Challenge Template

All challenges follow a four-phase template: setup, execute, assert, cleanup.

Code Block 8.8: Challenge Template — `selection_fairness_challenge.sh`

```

#!/bin/bash
set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../../../../" && pwd)"
COMPOSE_FILE="${PROJECT_ROOT}/tests/integration/test-compose.yml"
RESULTS_FILE="/tmp/selection_fairness_$.json"
EXPECTED_MAX_PERCENTAGE=15

setup() {
    echo "[SETUP] Starting environment..."
}

```

```

docker-compose -f "${COMPOSE_FILE}" up -d verifier mock-
    providers
for i in {1..30}; do
    curl -sf "http://localhost:18080/health" > /dev/null 2>&1
    && break
    sleep 1
done
echo "[SETUP] Seeding 10 equivalent models..."
for i in $(seq 1 10); do
    curl -sf -X POST "http://localhost:18080/api/v1/models" \
        -H "Content-Type: application/json" \
        -d "{\"model_id\":\"fair-model-${printf \"%03d\" $i}\",
        \"name\":\"Fair $i (SC:8.0)\",\"provider\":\"equiv-
        provider\", \"score\":8.0, \"status\":\"verified\"}" > /dev/
        null
done
}

execute() {
    echo "[EXECUTE] Running 1000 selections..."
    go run "${PROJECT_ROOT}/cmd/challenge-runner" selection-
        fairness \
        --verifier-url "http://localhost:18080" --model-count 10 \
        --request-count 1000 --output "${RESULTS_FILE}"
}

assert() {
    echo "[ASSERT] Validating distribution..."
    max_percentage=$(jq '.max_selection_percentage' "$
        {RESULTS_FILE}")
    selected_count=$(jq '.selected_models | length' "$
        {RESULTS_FILE}")

    if (( $(echo "${max_percentage} > ${EXPECTED_MAX_PERCENTAGE}"
        | bc -l) )); then
        echo "CHALLENGE FAILED: max percentage ${max_percentage}%
        exceeds ${EXPECTED_MAX_PERCENTAGE}%"
        exit 1
    fi
    if [[ "${selected_count}" -ne 10 ]]; then
        echo "CHALLENGE FAILED: only ${selected_count}/10 models
        selected"
        exit 1
    fi
    echo "CHALLENGE PASSED"
}

cleanup() {
    docker-compose -f "${COMPOSE_FILE}" down -v 2>/dev/null ||
        true
    rm -f "${RESULTS_FILE}"
}

trap cleanup EXIT

```

setup
execute
assert

The assert phase checks two anti-bluff conditions: no single model receives more than 15% (detecting a biased selector), and all 10 models receive at least one selection (detecting a selector that always picks the first item). `trap cleanup EXIT` ensures teardown even on assertion failure.

8.7.6 Challenge Registry

Table 8.3: All 193+ Challenges — Breakdown by Subsystem

Subsystem	Count	Pass Criteria	Execution Time	Required Resources
Scoring (S01–S20)	20	Computed score matches expected within 0.01 for each model archetype	~5 min	Dockerized LLMs + :memory: SQLite
Verification (V01–V80)	80	Each of 8 pipeline steps: 10 challenges for pass/fail/retry/timeout; full pipeline in 120s	~20 min	Dockerized LLMs + mock provider endpoint
Gatekeeping (G01–G12)	12	All 12 cells in 2×2×3 matrix correct; boundary at exact threshold	~2 min	:memory: SQLite
Discovery (D01–D30)	30	10 per tier; fallback ordering and timeout behavior	~10 min	Dockerized LLMs + mock providers
Selection Fairness (L01–L10)	10	Load within 15% max; minimum 3 providers in top-10	~8 min	Dockerized LLMs + 50+ mock providers
Score Adapter (A01–A05)	5	Field-by-field mapping matches expected JSON; cache correct	~3 min	Dockerized LLMs + real SQLite
Integration (I01–I20)	20	Full journey: discover→verify→score→select→translate→validate	~30 min	Full Docker Compose stack
Performance (P01–P16)	16	p50/p95/p99 within SLA; no goroutine leaks after 1h sustained	~90 min	4 CPU cores, 8GB RAM
Total	193	Zero diff in assertion phase	~168 min	Docker Compose CI runner

The 193 challenges distribute across nine subsystems, with the largest concentration (80) in the verification pipeline. Each challenge has a deterministic pass criterion. Performance challenges account for 90 of the 168 total minutes due to the 1-hour sustained load test.

8.8 Performance and Load Testing

8.8.1 Scoring Engine Performance Test

tests/perf/verifier_perf_test.go benchmarks the scoring engine under concurrent load using testing.B with b.ReportAllocs() and a custom errgroup-based load generator.

Code Block 8.9: tests/perf/verifier_perf_test.go

```
package perf

import (
    "context"
    "fmt"
    "sort"
    "sync"
    "testing"
    "time"

    "golang.org/x/sync/errgroup"
)

func BenchmarkScoringEngine_SingleScore(b *testing.B) {
    db := setupBenchmarkDB(b)
    defer cleanupBenchmarkDB(b, db)
    engine, mockClient := setupScoringEngine(b, db)
    seedBenchmarkModels(b, db, mockClient, 1)
    ctx := context.Background()
    config := DefaultScoreWeights()

    b.ResetTimer()
    b.ReportAllocs()
    for i := 0; i < b.N; i++ {
        _, err := engine.CalculateComprehensiveScore(ctx, "bench-model-000", config)
        if err != nil {
            b.Fatalf("scoring failed: %v", err)
        }
    }
}

func BenchmarkScoringEngine_BatchScore(b *testing.B) {
    db := setupBenchmarkDB(b)
    defer cleanupBenchmarkDB(b, db)
    engine, mockClient := setupScoringEngine(b, db)
    seedBenchmarkModels(b, db, mockClient, 100)
    ctx := context.Background()
    config := DefaultScoreWeights()
    modelIDs := make([]string, 100)
    for i := 0; i < 100; i++ {
        modelIDs[i] = fmt.Sprintf("bench-model-%03d", i)
    }
}
```

```

    }

    b.ResetTimer()
    b.ReportAllocs()
    for i := 0; i < b.N; i++ {
        _, err := engine.CalculateBatchScores(ctx, modelIDs,
            &config)
        if err != nil {
            b.Fatalf("batch scoring failed: %v", err)
        }
    }
}

}

func TestScoringEngine_ConcurrentLoad(t *testing.T) {
    if os.Getenv("RUN_PERF_TESTS") == "" {
        t.Skip("set RUN_PERF_TESTS=1")
    }
    db := setupBenchmarkDB(t)
    defer cleanupBenchmarkDB(t, db)
    engine, mockClient := setupScoringEngine(t, db)
    seedBenchmarkModels(t, db, mockClient, 50)

    ctx := context.Background()
    config := DefaultScoreWeights()
    const concurrentRequests = 1000

    latencies := make([]time.Duration, concurrentRequests)
    var latenciesMu sync.Mutex

    g, ctx := errgroup.WithContext(ctx)
    start := time.Now()
    for i := 0; i < concurrentRequests; i++ {
        idx := i
        modelID := fmt.Sprintf("bench-model-%03d", idx%50)
        g.Go(func() error {
            reqStart := time.Now()
            _, err := engine.CalculateComprehensiveScore(ctx,
                modelID, config)
            duration := time.Since(reqStart)
            latenciesMu.Lock()
            latencies[idx] = duration
            latenciesMu.Unlock()
            return err
        })
    }
    require.NoError(t, g.Wait())

    sort.Slice(latencies, func(i, j int) bool { return
        latencies[i] < latencies[j] })
    p50 := latencies[concurrentRequests*50/100]
    p95 := latencies[concurrentRequests*95/100]
    p99 := latencies[concurrentRequests*99/100]

```



```

t.Logf("1000 concurrent: total=%v p50=%v p95=%v p99=%v",
      time.Since(start), p50, p95, p99)
assert.Less(t, p50, 50*time.Millisecond, "p50 exceeds 50ms
      SLA")
assert.Less(t, p95, 200*time.Millisecond, "p95 exceeds 200ms
      SLA")
assert.Less(t, p99, 500*time.Millisecond, "p99 exceeds 500ms
      SLA")
}

```

b.ReportAllocs() detects memory allocation regressions. The concurrent load test collects 1000 latency samples, sorts them, and asserts against SLA targets from Table 8.4.

8.8.2 Performance SLA Targets

Table 8.4: Performance SLA Targets — Latency Percentiles by Operation

Operation	p50 Target	p95 Target	p99 Target	Benchmark Function	Failure Action
Score retrieval (single)	< 50ms	< 200ms	< 500ms	BenchmarkScoringEngine_SingleScore	CI build fails
Model listing (50 models)	< 100ms	< 300ms	< 1s	BenchmarkAPI_ListModels	CI build fails
Full verification (8 steps)	< 5s	< 30s	< 60s	TestPipeline_FullVerification	CI build fails
Discovery (single provider)	< 10s	< 30s	< 60s	TestDiscovery_SingleProvider	CI build fails
Model selection (cached)	< 10ms	< 50ms	< 100ms	BenchmarkSelectionEngine_Select	CI build fails
Batch calculation (100)	< 3s	< 10s	< 20s	BenchmarkScoringEngine_BatchScore	Warning alert
Concurrent retrieval (1000)	< 50ms	< 200ms	< 500ms	TestScoringEngine_ConcurrentLoad	CI build fails

8.8.3 Memory Leak Detection

The 1-hour sustained load test runs 100 workers making score requests every 100ms, sampling heap and goroutine counts at 0, 30, and 60 minutes.

Code Block 8.10: Memory Leak Detection Test

```
package perf
```

```

import (
    "context"
    "fmt"
    "os"
    "runtime"
    "runtime/pprof"
    "testing"
    "time"

    "golang.org/x/sync/errgroup"
)

func TestSustainedLoad_OneHour(t *testing.T) {
    if os.Getenv("RUN_SUSTAINED_TEST") == "" {
        t.Skip("set RUN_SUSTAINED_TEST=1")
    }

    db := setupBenchmarkDB(t)
    defer cleanupBenchmarkDB(t, db)
    engine, mockClient := setupScoringEngine(t, db)
    seedBenchmarkModels(t, db, mockClient, 50)

    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()
    config := DefaultScoreWeights()

    heapProfiles := make([]*runtime.MemStats, 3)
    goroutineCounts := make([]int, 3)
    sampleTimes := []time.Duration{0, 30 * time.Minute, 60 *
        time.Minute}

    g, ctx := errgroup.WithContext(ctx)
    loadStart := time.Now()
    for i := 0; i < 100; i++ {
        workerID := i
        g.Go(func() error {
            ticker := time.NewTicker(100 * time.Millisecond)
            defer ticker.Stop()
            reqCount := 0
            for {
                select {
                case <-ctx.Done():
                    t.Logf("Worker %d: %d requests", workerID,
                        reqCount)
                    return nil
                case <-ticker.C:
                    modelID := fmt.Sprintf("bench-model-%03d",
                        reqCount%50)
                    if _, err :=
                        engine.CalculateComprehensiveScore(ctx, modelID, config);
                    err != nil {
                        return err
                    }
                }
            }
        })
    }
}

```

```

        reqCount++
    }
}
}))
}

for sampleIdx, delay := range sampleTimes {
    time.Sleep(delay - time.Since(loadStart))
    runtime.GC()
    time.Sleep(2 * time.Second)
    var m runtime.MemStats
    runtime.ReadMemStats(&m)
    heapProfiles[sampleIdx] = &m
    goroutineCounts[sampleIdx] = runtime.NumGoroutine()
    profilePath := fmt.Sprintf("/tmp/
    heap_profile_%d_min.prof", int(delay.Minutes()))
    f, _ := os.Create(profilePath)
    pprof.WriteHeapProfile(f)
    f.Close()
    t.Logf("Sample %d: heap=%dMB goroutines=%d", sampleIdx,
    m.HeapAlloc/1024/1024, goroutineCounts[sampleIdx])
}

cancel()
require.NoError(t, g.Wait())

heapGrowth :=
    float64(heapProfiles[2].HeapAlloc-heapProfiles[1].HeapAlloc) /
    float64(heapProfiles[1].HeapAlloc) * 100
assert.Less(t, heapGrowth, 10.0, "heap grew %.2f%% in second
    30min - possible leak", heapGrowth)

goroutineGrowth := goroutineCounts[2] - goroutineCounts[0]
assert.LessOrEqual(t, goroutineGrowth, 5, "goroutines grew by
    %d - possible leak", goroutineGrowth)
}

```

The test forces GC before each sample for consistent measurements. Anti-bluff checks assert heap growth under 10% in the second 30-minute period and goroutine growth under 5. Heap profiles are preserved as artifacts for post-hoc analysis. The `cancel()` followed by `g.Wait()` ensures all workers exit before the final sample, preventing the test itself from appearing as a leak source.